

Estructura y Arquitectura de Software

R2-A2-S15
Bases de datos

Rodrigo Alberto Osorio Gaona
Sebastian David Gonzales Hernandez

Universidad de Cundianamarca
Ingeniería de Sistemas y Computación
Colombia
2026

1. Introducción y Contexto del Sistema

1.1 Descripción del Dominio

El sistema Ebook es una plataforma de comercio electrónico orientada al intercambio y compraventa de libros físicos y digitales entre usuarios. El repositorio analizado implementa una arquitectura de microservicios en Java 17 con Spring Boot 3.5.x, siguiendo principios de arquitectura limpia (Clean/Hexagonal Architecture) con separación de capas dominio, aplicación e infraestructura.

El negocio cubre tres flujos principales: (1) gestión del inventario de libros por parte de los usuarios-vendedores, (2) compraventa transaccional con procesamiento de pagos, y (3) sistema de intercambio de libros entre usuarios mediante publicaciones y ofertas negociadas.

1.2 Microservicios Identificados

Microservicio	Puerto	Artefacto Maven	Responsabilidad
carrito-service	8081	carrito-service	Libros, Transacciones, Intercambios, Ventas, Compras
usuario-service	9091	productos	Gestión de usuarios, autenticación, cifrado
pago-service	9093	pago	Procesamiento y reversión de pagos, referencias de pago

1.3 Supuestos Arquitectónicos

- El sistema se asume en fase de MVP/crecimiento temprano con proyección a escala empresarial.
- Se proyectan 50,000 usuarios activos al año 1, con crecimiento del 100% anual durante los primeros 3 años.
- El volumen de catálogo de libros se estima en 500,000 registros en el año 2.
- El servicio de pago es actualmente una simulación; se asume integración futura con pasarelas reales (Stripe, PayU, MercadoPago).
- Los microservicios están desplegados en un único host (localhost) sin orquestador ni API Gateway; esto es una limitación de desarrollo, no de producción.

1.4 Problemas Arquitectónicos Detectados

- CRÍTICO: Los 3 microservicios comparten la misma base de datos PostgreSQL (biblioteca), violando el principio de database-per-service de microservicios.
- CRÍTICO: Conflictos de merge de Git sin resolver en IntercambioController.java, indicando ramas divergentes en producción.
- ALTO: Las contraseñas se almacenan y comparan sin bcrypt en el flujo de login del UsuarioController.
- ALTO: El endpoint /login retorna el objeto Usuario completo incluyendo el campo password sobre HTTP.
- MEDIO: No existe API Gateway, service discovery, ni circuit breaker (Resilience4j/Hystrix).
- MEDIO: Comunicación inter-servicios vía RestTemplate síncrono, sin tolerancia a fallos distribuidos.
- MEDIO: Ausencia de capa de caché, message broker y motor de búsqueda full-text.

- BAJO: La búsqueda de libros por título y autor se realiza con LIKE en JPA, ineficiente a escala.

2. Fase 1 — Análisis de Requisitos de Datos

2.1 Clasificación de Entidades del Dominio

A continuación se clasifican las nueve entidades identificadas en el modelo de dominio según sus características de acceso, volumen y requisitos de consistencia:

Entidad	Volumen Est.	Freq. Lectura	Freq. Escritura	Consistencia	Sensibilidad	Motor Sugerido
Usuario	Alto	Muy Alta	Media	Fuerte	Alta	PostgreSQL
Libro	Muy Alto	Muy Alta	Media	Fuerte	Baja	PostgreSQL + Elasticsearch + Redis
Transaccion	Alto	Alta	Alta	Fuerte ACID	Crítica	PostgreSQL
Pago	Alto	Media	Alta	Fuerte ACID	Crítica	PostgreSQL
Compra / Venta	Alto	Alta	Media	Fuerte	Media	PostgreSQL
Intercambio	Medio	Alta	Alta	Fuerte	Baja	PostgreSQL
PublicacionIntercambio	Medio	Muy Alta	Media	Eventual	Baja	PostgreSQL + Redis (cache)
OfertaIntercambio	Medio	Alta	Media	Eventual	Baja	PostgreSQL
Sesión / Token	Alto	Muy Alta	Muy Alta	Eventual BASE	Alta	Redis

2.2 Clasificación por Necesidad de ACID vs BASE

Entidades ACID (requieren transaccionalidad fuerte): Transaccion, Pago, Compra, Venta, Usuario. Son operaciones financieras y de identidad donde la atomicidad y durabilidad son no negociables. Un fallo parcial en la creación de una Transaccion junto con su Pago podría dejar el sistema en estado inconsistente con implicaciones regulatorias y de negocio.

Entidades BASE (toleran consistencia eventual): Sesiones de usuario, catálogos en caché, publicaciones de intercambio visibles públicamente. La latencia de replicación de 50-200ms es aceptable para la experiencia de lectura del catálogo.

2.3 Justificación Técnica de Motores por Grupo de Entidades

PostgreSQL — Datos transaccionales: Todos los datos que requieren garantías ACID, integridad referencial y consultas relacionales complejas (JOINS entre Transaccion-Libro-Usuario). PostgreSQL ofrece MVCC, WAL, soporte nativo a JSON, full-text search básico, y es el motor más maduro para cargas OLTP mixtas. Su JSONB permite flexibilidad estructural sin sacrificar consistencia.

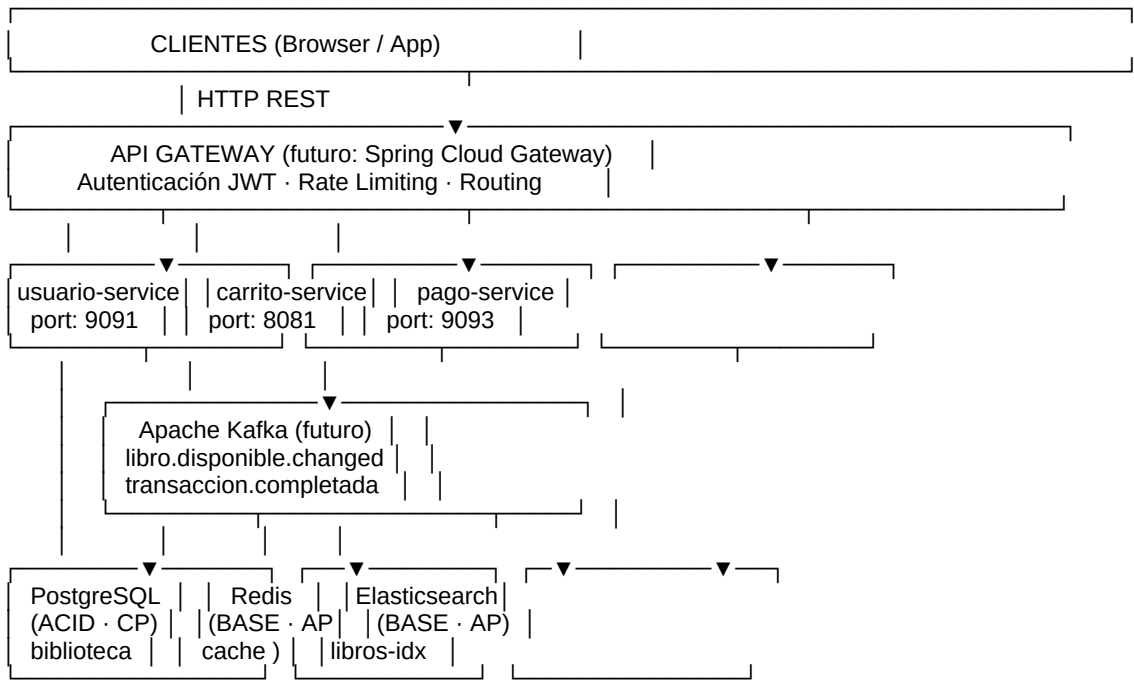
Redis — Caché y sesiones: Las consultas de libros disponibles son la operación más frecuente del sistema (lectura del catálogo). Redis permite cachear este conjunto con TTL configurable, reduciendo la carga sobre PostgreSQL en un estimado del 60-70%. Adicionalmente, gestiona tokens de sesión JWT con expiración automática, datos de rate-limiting y contadores de stock en memoria con operaciones atómicas INCR/DECR.

Elasticsearch — Búsqueda full-text: La funcionalidad de búsqueda por título y autor actualmente usa LIKE '%texto%' en JPA, lo que impide el uso de índices en PostgreSQL y escala de forma O(N). Elasticsearch resuelve esto con índices invertidos, análisis léxico (stemming en español), búsqueda difusa (fuzzy) y relevancia por TF-IDF. Para 500,000 libros, la diferencia es de 200-500ms (PostgreSQL LIKE) a <10ms (Elasticsearch).

3. Fase 2 — Arquitectura Polyglot Persistence

3.1 Diagrama de Arquitectura

La arquitectura propuesta organiza los tres motores de persistencia en capas con responsabilidades claras. A continuación se presenta la descripción del diagrama:



3.2 Ownership de Datos y Bounded Contexts

Bounded Context	Entidades	Motor Primario	Justificación
Identidad y Acceso	Usuario, Sesión	PostgreSQL + Redis	Datos de identidad en PG (ACID). Tokens JWT y sesiones en Redis con TTL de 24h.
Catálogo de Productos	Libro	PostgreSQL + Redis + ES	Source of truth en PG. Cache de catálogo público en Redis. Índice de búsqueda en Elasticsearch.
Gestión Comercial	Transaccion, Compra, Venta	PostgreSQL	Requiere ACID estricto. JOINS complejos entre entidades relacionadas.
Pagos	Pago	PostgreSQL	Máxima consistencia financiera. Trazabilidad completa con fechas y referencias.
Mercado de Intercambio	Intercambio, Publicacion, Oferta	PostgreSQL + Redis	Publicaciones activas cacheadas en Redis. Estado de ofertas en

			PG.
--	--	--	-----

3.3 Mecanismos de Sincronización entre Motores

Patrón Write-Through para Redis: Cuando un Libro se actualiza en PostgreSQL, el Gateway de infraestructura invalida la entrada correspondiente en Redis (cache-aside con invalidación activa). No se usa write-through directo para evitar latencia adicional en escrituras críticas.

Patrón Outbox + Kafka para Elasticsearch: Al crear o actualizar un Libro, se escribe un evento en una tabla outbox en la misma transacción PostgreSQL. Un proceso Change Data Capture (Debezium) lee el WAL de PostgreSQL y publica el evento en Kafka. Un consumidor indexa el documento en Elasticsearch. Esto garantiza que el índice de búsqueda eventualmente refleja el estado de la base de datos sin acoplar la escritura principal.

Transacciones Distribuidas — Patrón Saga: El flujo crearTransaccionCompra involucra múltiples servicios. Se recomienda el patrón Saga Coreografiada mediante eventos Kafka: el carrito-service emite TransaccionCreada, el pago-service consume y emite PagoCompletado o PagoFallido, y el carrito-service reacciona actualizando el estado. Esto elimina el acoplamiento síncrono actual vía RestTemplate.

3.4 Matriz de Decisión Tecnológica

Criterio	PostgreSQL	Redis	Elasticsearch	MongoDB	Decisión
Consistencia ACID	✓✓✓	Parcial	✗	Parcial	PG para ACID
Búsqueda full-text	Básica	✗	✓✓✓	✓	ES para búsqueda
Latencia lectura (<5ms)	~10ms	✓✓✓ <1ms	~5ms	~10ms	Redis para caché
Costo operativo	Medio	Bajo	Medio-Alto	Medio	PG+Redis primero
Madurez ecosistema	✓✓✓	✓✓✓	✓✓	✓✓	—
Spring Boot integración	✓✓✓	✓✓✓	✓✓	✓✓	—

4. Fase 3 — Modelado Físico y Patrones de Acceso

4.1 Modelo Relacional PostgreSQL (ERD Textual)

El siguiente esquema representa el modelo físico propuesto con separación de esquemas por bounded context:

Schema: identity — Bounded Context de Identidad

```
CREATE TABLE identity.usuario (
  id      BIGSERIAL PRIMARY KEY,
  nombre  VARCHAR(150) NOT NULL,
  email   VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL, -- bcrypt, NUNCA texto plano
  activo  BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_usuario_email ON identity.usuario(email);
```

Schema: catalog — Bounded Context de Catálogo

```
CREATE TABLE catalog.libro (  
  id_libro BIGSERIAL PRIMARY KEY,  
  titulo VARCHAR(500) NOT NULL,  
  autor VARCHAR(300) NOT NULL,  
  descripcion TEXT,  
  precio NUMERIC(12,2) NOT NULL CHECK (precio >= 0),  
  stock INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),  
  estado VARCHAR(20) NOT NULL CHECK (estado IN ('NUEVO','USADO','DIGITAL')),  
  usuario_id BIGINT NOT NULL REFERENCES identity.usuario(id),  
  disponible BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE INDEX idx_libro_usuario ON catalog.libro(usuario_id);  
CREATE INDEX idx_libro_disponible ON catalog.libro(disponible) WHERE disponible = TRUE;  
CREATE INDEX idx_libro_titulo_trgm ON catalog.libro USING gin(titulo gin_trgm_ops);
```

Schema: commerce — Bounded Context Comercial

```
CREATE TABLE commerce.transaccion (  
  id_transaccion BIGSERIAL PRIMARY KEY,  
  libro_id BIGINT NOT NULL REFERENCES catalog.libro(id_libro),  
  comprador_id BIGINT NOT NULL REFERENCES identity.usuario(id),  
  vendedor_id BIGINT NOT NULL REFERENCES identity.usuario(id),  
  precio NUMERIC(12,2) NOT NULL,  
  cantidad INTEGER NOT NULL DEFAULT 1,  
  estado VARCHAR(30) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
CREATE INDEX idx_transaccion_comprador ON commerce.transaccion(comprador_id);  
CREATE INDEX idx_transaccion_vendedor ON commerce.transaccion(vendedor_id);  
CREATE INDEX idx_transaccion_estado ON commerce.transaccion(estado);
```

4.2 Modelo de Documentos Redis

Redis almacena datos estructurados como Hash o como JSON serializado. Los patrones de clave siguen la convención namespace:entidad:id:

```
# Caché de libro individual (TTL: 10 minutos)  
HSET catalog:libro:1001 titulo "Clean Code" autor "Robert C. Martin"  
  precio "45.00" stock "12" disponible "true"  
EXPIRE catalog:libro:1001 600
```

```
# Set de IDs de libros disponibles — para listados rápidos (TTL: 5 min)  
SADD catalog:libros:disponibles 1001 1002 1003 1045  
EXPIRE catalog:libros:disponibles 300
```

```
# Sesión de usuario con JWT payload (TTL: 24 horas)  
HSET session:usuario:42 jwt_token "eyJ..." email "user@mail.com" nombre "Ana"  
EXPIRE session:usuario:42 86400
```

```
# Stock en tiempo real — operaciones atómicas  
SET catalog:stock:1001 12  
DECR catalog:stock:1001 -- Al confirmar transacción
```

4.3 Documento Elasticsearch — Índice de Libros

El mapping del índice libros-idx en Elasticsearch está optimizado para búsqueda en español:

```
{
```

```

"mappings": {
  "properties": {
    "id_libro": { "type": "long" },
    "titulo": { "type": "text", "analyzer": "spanish", "boost": 3.0 },
    "autor": { "type": "text", "analyzer": "spanish", "boost": 2.0 },
    "descripcion": { "type": "text", "analyzer": "spanish" },
    "precio": { "type": "scaled_float", "scaling_factor": 100 },
    "disponible": { "type": "boolean" },
    "estado": { "type": "keyword" },
    "usuario_id": { "type": "long" }
  }
}
}

```

4.4 Patrones de Acceso Principales

Patrón 1: Búsqueda de libros por título o autor (alta frecuencia)

Caso de uso: Un usuario escribe 'robert martin' en el buscador para encontrar libros disponibles.

Métricas: Latencia objetivo < 50ms · Concurrencia estimada: 200 req/s · Volumen: 500,000 documentos

Solución actual (JPA LIKE — ineficiente):

```

SELECT * FROM libro WHERE LOWER(titulo) LIKE LOWER('%robert martin%')
-- Full table scan, no usa índices, escala O(N)

```

Solución propuesta (Elasticsearch):

```

{
  "query": {
    "bool": {
      "must": [{ "term": { "disponible": true } }],
      "should": [
        { "match": { "titulo": { "query": "robert martin", "fuzziness": "AUTO" } }},
        { "match": { "autor": { "query": "robert martin", "boost": 2.0 } }}
      ]
    }
  }
}

```

Por qué funciona: índice invertido con análisis léxico en español. Busca tokens normalizados, acepta errores tipográficos (fuzziness) y pondera por campo (título > autor). Latencia: <10ms vs ~300ms de LIKE.

Patrón 2: Listado de libros disponibles (catálogo público)

Caso de uso: La página principal muestra todos los libros disponibles para compra o intercambio.

Métricas: Latencia objetivo < 5ms · Concurrencia: 500 req/s · Datos prácticamente estáticos entre escrituras

Flujo con Redis Cache-Aside:

1. Consultar Redis: SMEMBERS catalog:libros:disponibles
2. Si HIT: Iterar IDs → HGETALL catalog:libro:{id} para cada uno → respuesta en <2ms
3. Si MISS: SELECT * FROM catalog.libro WHERE disponible = TRUE
→ Poblar Redis (SADD + HSET por cada libro) → EXPIRE 300
4. Invalidación: al ejecutar PUT /publicar/{id} → DEL catalog:libros:disponibles

Por qué funciona: Redis opera en memoria con O(1) para HGETALL y O(N) para SMEMBERS donde N es el tamaño del set, no el tamaño del catálogo completo. Hit ratio estimado >80% dado el patrón de lectura pesada.

Patrón 3: Creación de transacción de compra (escritura crítica)

Caso de uso: Un usuario compra un libro. El flujo requiere validar usuario, libro, stock, procesar pago y registrar la transacción de forma atómica.

Métricas: Latencia objetivo < 500ms · Concurrencia: 50 req/s · Criticidad: MÁXIMA (financiero)

Consulta SQL con bloqueo pesimista para control de stock:

```
BEGIN;
-- Bloqueo de fila para evitar race condition en stock
SELECT id_libro, stock, precio, usuario_id FROM catalog.libro
  WHERE id_libro = $1 AND disponible = TRUE FOR UPDATE;
-- Verificar stock >= cantidad solicitada
INSERT INTO commerce.transaccion (libro_id, comprador_id, vendedor_id, precio, cantidad, estado)
  VALUES ($1, $2, $3, $4, $5, 'PENDIENTE_PAGO') RETURNING id_transaccion;
-- Llamar al servicio de pago (resultado en $pago_ok)
UPDATE commerce.transaccion SET estado = CASE WHEN $pago_ok THEN 'PENDIENTE_CONFIRMACION'
  ELSE 'PAGO_FALLIDO' END WHERE id_transaccion = $6;
COMMIT;
```

Por qué funciona: FOR UPDATE garantiza que dos compradores no agoten el mismo stock simultáneamente. La transacción PostgreSQL asegura que si el pago falla, el registro de transacción se actualiza correctamente (sin rollback total, para preservar la auditoría).

5. Fase 4 — Análisis CAP y Estrategia de Escalabilidad

5.1 Posicionamiento CAP por Motor

Motor	Consistencia	Disponibilidad	Part. Tolerance	Implicación para Ebook
PostgreSQL	✓✓✓ Fuerte	Alta (no máxima)	Limitada (CA)	Garantiza integridad de transacciones financieras. En partición de red, prioriza consistencia sobre disponibilidad — aceptable para operaciones de pago.
Redis	Eventual (Sentinel)	✓✓✓ Muy Alta	✓✓✓ Alta	Caché tolerante a fallos. Si Redis cae, el sistema degrada a consultar PostgreSQL directamente. Sesiones expiran naturalmente.
Elasticsearch	Eventual	✓✓✓ Muy Alta	✓✓✓ Alta	El índice de búsqueda puede estar momentáneamente desactualizado (segundos). Aceptable: la búsqueda no es operación financiera.

5.2 Estrategias de Escalabilidad por Capa

PostgreSQL — Escalabilidad Vertical + Réplicas de Lectura:

- Fase 1 (0-50k usuarios): Una instancia con 16 vCPUs, 64GB RAM. Estimado: 1,000 TPS con índices optimizados.
- Fase 2 (50k-200k usuarios): Read replicas para cargas analíticas y consultas de historial (streaming replication). Writes siempre al primario.
- Fase 3 (>200k usuarios): Sharding por usuario_id usando Citus (PostgreSQL distribuido) o migración a Aurora PostgreSQL con auto-scaling.
- Particionado de tabla transaccion por rango de fecha (PARTITION BY RANGE created_at) para optimizar consultas históricas y archivado.

Redis — Cluster Mode:

- Inicio: Redis Sentinel con 1 primario + 2 réplicas para HA. Failover automático en <30 segundos.
- Escala: Redis Cluster con 3 shards (6 nodos), distribución automática por hash slot. Soporta hasta 1M operaciones/segundo.
- Eviction policy: allkeys-lru para el caché de catálogo. noeviction para sesiones (datos críticos).

Elasticsearch — Cluster Multi-nodo:

- Inicio: 3 nodos (1 master + 2 data nodes), 1 réplica por shard. Estimado: 10M documentos, <10ms p99.
- Escala: índices con 5 primary shards, ajuste dinámico de réplicas según carga. ILM (Index Lifecycle Management) para archivar documentos inactivos.

Microservicios — Escalabilidad Horizontal:

- Contenerización Docker + orquestación Kubernetes. Cada servicio con HPA (Horizontal Pod Autoscaler) basado en CPU y métricas custom (req/s).

- carrito-service: escala a 5-10 réplicas en peak (búsquedas + transacciones).
- pago-service: 2-3 réplicas con rate limiting estricto para evitar procesamiento duplicado.
- usuario-service: 2-3 réplicas con sticky sessions opcionales para optimizar caché local.

5.3 Estimaciones de Capacidad

Métrica	Año 1	Año 2	Año 3	Observación
Usuarios activos	50,000	120,000	250,000	Crecimiento 100%/año proyectado
Libros en catálogo	200,000	500,000	1,200,000	~5 libros/usuario activo promedio
TPS promedio	50 TPS	150 TPS	400 TPS	Peak 3x promedio en horario pico
Transacciones/día	5,000	15,000	40,000	~10% usuarios realizan transacción/día
Almacenamiento PG	~50 GB	~150 GB	~400 GB	Incluyendo WAL e índices
Almacenamiento ES	~10 GB	~25 GB	~60 GB	Documentos compactos, sin imágenes
RAM Redis	4 GB	12 GB	32 GB	Catálogo + sesiones + rate limiting

6. Fase 5 — Seguridad, Gobierno y Observabilidad

6.1 Matriz de Seguridad

Área	Problema Actual	Solución Propuesta	Justificación
Autenticación	Contraseña en texto plano en login	JWT + bcrypt (cost factor 12)	bcrypt con cost 12 produce hash irreversible en ~250ms (suficiente para prevenir brute-force). JWT stateless, firmado con RS256.
Autorización	Verificación manual de usuariold en controladores	Spring Security + RBAC (roles: ADMIN, VENDEDOR, COMPRADOR)	@PreAuthorize en métodos de negocio. Elimina verificaciones manuales propensas a errores. Permite expansión a ABAC futuro.
Exposición de datos	Login retorna objeto Usuario con password	DTO LoginResponse sin campo password. Uso de @JsonIgnore.	El password_hash nunca debe salir de la capa de dominio. DTOs separados para requests y responses.
Cifrado en tránsito	HTTP (sin TLS observado)	TLS 1.3 obligatorio en API Gateway. Certificados Let's Encrypt o ACM.	TLS 1.3 reduce handshake a 1-RTT. Obligatorio para cumplimiento PCI-DSS en flujos de pago.
Cifrado en reposo	No configurado	PostgreSQL: Transparent Data Encryption. Redis: ACL + TLS. ES: SSL + field-level encryption para datos	Datos de usuario (email, nombre) son PII bajo GDPR/Ley 1581 Colombia.

		PII.	
Gestión de secretos	Contraseñas en application.properties en Git	HashiCorp Vault o AWS Secrets Manager. Spring Cloud Vault para inyección en runtime.	NUNCA credenciales en el repositorio. Rotación automática de claves cada 90 días.
CORS	@CrossOrigin hardcodeado en controladores	Configuración CORS centralizada en API Gateway con whitelist de dominios.	Evita regresiones de seguridad al agregar endpoints. Control centralizado.

6.2 Gobierno de Datos

Práctica	Implementación	Herramienta
Auditoría	Tabla commerce.auditoria_transaccion con trigger PostgreSQL: registra usuario, acción, timestamp, IP, valor anterior y nuevo.	PostgreSQL triggers + Audit Log
Retención de datos	Transacciones: 7 años (requisito fiscal Colombia). Logs de acceso: 1 año. Datos de sesión: 24h (Redis TTL).	PG Partitioning + pg_cron para archivado
Backup	PostgreSQL: pg_basebackup diario + WAL archiving continuo (PITR). Redis: RDB cada hora + AOF. Elasticsearch: snapshot a S3 cada 6h.	Barman / pgBackRest
Clasificación PII	Campos PII identificados: email, nombre, dirección de entrega. Aplicar pseudonimización y controles de acceso diferenciados.	PostgreSQL RLS (Row Level Security)
RPO / RTO	RPO objetivo: 15 minutos (pérdida máxima aceptable de datos). RTO objetivo: 1 hora (tiempo máximo de restauración).	Multi-AZ deployment + PITR

6.3 Plan de Observabilidad — Pila Propuesta

Se propone la pila de observabilidad estándar de la industria (OpenTelemetry + Grafana Stack):

- Métricas: Micrometer (Spring Boot Actuator) → Prometheus → Grafana. Dashboards: latencia P50/P95/P99 por endpoint, TPS, error rate, pool de conexiones JDBC, hit ratio de Redis, tamaño de índice Elasticsearch.
- Logs: Logback → ELK Stack (Elasticsearch + Logstash + Kibana) o OpenSearch. Formato JSON estructurado con correlationId, userId, y traceId. Alertas por patrón de errores (5xx > 1% en 5 minutos).
- Tracing distribuido: OpenTelemetry SDK (Spring Boot auto-instrumentation) → Jaeger o Zipkin. Cada request HTTP genera un traceId propagado a través de todos los microservicios. Esencial para diagnosticar la cadena carrito → pago → confirmación.
- Alertas críticas: Prometheus Alertmanager. Disparadores: transacción sin confirmación en >10 min, error de pago >5% en 1 min, stock negativo detectado (invariante rota), latencia P99 >1s en /api/ecommerce/transacciones.
- Monitoreo de replicación: pg_stat_replication para réplicas PostgreSQL. Alerta si replication_lag > 30 segundos.

7. Riesgos Técnicos

Riesgo	Probabilidad	Impacto	Mitigación
Base de datos compartida entre microservicios	Alta	Crítico	Migrar a database-per-service. Usar schemas separados como paso intermedio antes de separar instancias.
Race condition en stock de libros	Media	Alto	SELECT FOR UPDATE en PostgreSQL. Implementar contador atómico en Redis como primera línea de defensa.
Inconsistencia ES vs PostgreSQL	Alta	Medio	Patrón Outbox + CDC (Debezium). Monitorear lag de sincronización. Permitir re-indexación completa manual.
Pago simulado sin idempotencia	Alta en producción	Crítico	Implementar idempotency keys en pago-service. Tabla de idempotency_keys con UNIQUE(key) para evitar cobros duplicados.
Sin circuit breaker entre servicios	Media	Alto	Implementar Resilience4j. Si pago-service no responde, retornar error 503 limpio en lugar de timeouts en cascada.
Credenciales en repositorio Git	Ya ocurrido	Crítico	Rotar INMEDIATAMENTE credenciales de PostgreSQL. Implementar git-secrets en pre-commit hook. Migrar a Vault.

8. Conclusiones Técnicas

El sistema Ebook presenta una base arquitectónica sólida en términos de separación de capas (Clean Architecture) y definición de dominio de negocio. Sin embargo, en su estado actual, presenta limitaciones críticas que deben abordarse antes de cualquier despliegue productivo.

Arquitectura polyglot recomendada: La combinación PostgreSQL (source of truth ACID) + Redis (caché y sesiones) + Elasticsearch (búsqueda full-text) cubre el 95% de los patrones de acceso identificados de forma óptima. Esta arquitectura es alcanzable de forma incremental: comenzar con PostgreSQL + Redis en el corto plazo, incorporar Elasticsearch en el mediano plazo a medida que el volumen de catálogo supere los 50,000 libros.

Prioridad de correcciones: (1) Credenciales en Git — acción inmediata. (2) Cifrado de contraseñas con bcrypt. (3) Separación de base de datos por microservicio usando schemas como etapa transitoria. (4) Eliminación de conflictos de merge. (5) Implementación de idempotencia en pagos.

Escalabilidad: La arquitectura propuesta soporta crecimiento de 50,000 a 250,000 usuarios sin refactoring mayor, únicamente mediante escalado horizontal de contenedores y ampliación de clusters de Redis y Elasticsearch. El cuello de botella proyectado está en PostgreSQL alrededor de los 200,000 usuarios activos, donde se deberá considerar Citus o Aurora PostgreSQL.

Observabilidad: La ausencia total de tracing distribuido y métricas de aplicación en el sistema actual imposibilita el diagnóstico de problemas en producción. La implementación de OpenTelemetry + Grafana debe considerarse requisito previo al despliegue, no una mejora posterior.

9. Referencias

- PostgreSQL 16 Documentation — <https://www.postgresql.org/docs/16/>
- Redis Documentation — <https://redis.io/docs/>

- Elasticsearch Guide 8.x — <https://www.elastic.co/guide/en/elasticsearch/reference/current/>
- Spring Boot Reference — <https://docs.spring.io/spring-boot/docs/3.5.x/reference/html/>
- Designing Data-Intensive Applications — Martin Kleppmann, O'Reilly 2017
- Building Microservices (2nd ed.) — Sam Newman, O'Reilly 2021
- Microservices Patterns — Chris Richardson, Manning 2018
- CAP Theorem Revisited — Eric Brewer, 2012 — <https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>
- Debezium CDC Documentation — <https://debezium.io/documentation/>
- OWASP Top 10 — <https://owasp.org/www-project-top-ten/>
- Ley 1581 de 2012 Colombia (Habeas Data) — Congreso de la República
- PCI-DSS v4.0 — Payment Card Industry Security Standards Council